

Clean Architecture ile İK Yönetim Uygulaması

Backend: .NET 10 · ASP.NET Core | Veri erişimi: Dapper | Kimlik: JWT | Mimari: Clean Architecture

1. Projenin Amacı ve Kapsamı

Bu projenin amacı, şirketin İnsan Kaynakları (İK) departmanının temel ihtiyaçlarını karşılayabilecek, web arayüzüne sahip, Clean Architecture prensiplerine uygun bir İK yönetim uygulaması geliştirmektir.

2. Teknoloji ve Genel Kurallar

Backend Teknolojisi	.NET 10 · ASP.NET Core (tercihen Web API + basit bir MVC/Razor/Blazor UI)
Veritabanı	MSSQL veya Mongo · ORM: Dapper
Kimlik Doğrulama	JWT tabanlı kimlik doğrulama
Arayüz (Frontend)	ASP.NET Core MVC / Razor Pages · Basit HTML/CSS/Bootstrap kullanılabilir
Versiyon Kontrolü	Git kullanılmalı; GitHub / GitLab / Bitbucket üzerinde repo tutulmalı
Dokümantasyon	README · Mimariyi ve katmanları anlatan kısa doküman · Veritabanı şeması

3. Mimari Gereksinimler (Clean Architecture)

Uygulama, temel Clean Architecture prensiplerine göre en az aşağıdaki proje/katman yapısına sahip olmalıdır.

3.1. Proje Yapısı

Çözüm içinde en az şu projeler bulunmalıdır:

1. ProjectName.Domain

- Temel Entity'ler (ör. Employee, Department, LeaveRequest, Intern)
- İş kurallarına ait temel metotlar (mümkün olduğunca anemik modelden kaçınma)
- Hiçbir dış bağımlılık olmamalı (EF Core, UI, Infrastructure vb. referansı YOK)

2. ProjectName.Application

- Use Case'ler / iş mantığı (ör. CreateEmployee, ApproveLeaveRequest, CreateIntern)
- DTO, Command/Query modelleri (isteğe bağlı basit bir CQRS yaklaşımıyla)
- Domain'e ait interface'ler (ör. IEmployeeRepository, ILeaveRequestRepository)
- Validasyon kuralları (ör. FluentValidation ya da manuel)
- Bağımlılık: sadece Domain katmanına referans verebilir

3. ProjectName.Infrastructure

- Dapper ile repository implementasyonları (ör. EmployeeRepository : IEmployeeRepository)
- Veritabanı bağlantı ayarları
- Varsa dış servisler için implementasyonlar
- Bağımlılık: Domain ve Application'a referans verebilir, Presentation'ı referans ALMAZ

4. ProjectName.Presentation (WebUI / API)

- ASP.NET Core web uygulaması (MVC, Razor Pages)
- Controller'lar / Pages / Components
- Login / Register / Authorization uçları
- Kullanıcıya dönen View / JSON sonuçları
- Bağımlılık: Application (ve dolaylı olarak Domain); Infrastructure'ı DI ile ekler (Program.cs)

3.2. Bağımlılık Kuralları

- Domain katmanı hiçbir katmana bağımlı olmayacak.
- Application katmanı sadece Domain katmanına bağımlı olacak.
- Infrastructure katmanı Domain + Application'a bağımlı olabilir.
- Presentation katmanı, Application (ve dolaylı olarak Domain) katmanına bağımlı olacak.
- İş kuralları (ör. izin onay akışı) Presentation'da değil, Application/Domain içinde olmalı.

3.3. Katmanlar Arası İletişim

- Akış: Controller (Presentation) → Application (Use Case / Service) → Domain Entity'leri → Infrastructure (Repository).
- Repository'ler, Application katmanında tanımlanan interface'ler üzerinden kullanılmalıdır.
- Dependency Injection ile: IEmployeeRepository gibi interface'ler Infrastructure'daki implementasyonlara bağlanır.

4. Kullanıcı Roller

- 1 Admin
- 2 HR (İK Uzmanı)
- 3 Yönetici (Manager)
- 4 Çalışan / Stajyer

5. Fonksiyonel Gereksinimler

5.1. Kimlik Doğrulama ve Yetkilendirme

- Kullanıcılar e-posta veya kullanıcı adı + şifre ile giriş yapabilmelidir.
- Şifreler veritabanında şifrelenmiş / hash'lenmiş olarak tutulmalıdır.
- Her kullanıcının bir rolü olmalıdır (Admin, HR, Yönetici, Çalışan).

- Admin ekranlarına sadece Admin, HR ekranlarına sadece HR, Yönetici ekranlarına ilgili rol erişebilir.
- Çalışan ekranında, giriş yapmış kullanıcı yalnızca kendi bilgilerini görebilir.

5.2. Çalışan Yönetimi (HR Modülü)

HR rolü için minimum gereksinimler:

- **Çalışan listesi ekranı:** İsim, Soyisim, Departman, Pozisyon, E-posta, Durum (Aktif/Pasif).
- **Çalışan ekleme/güncelleme ekranı:** Ad, Soyad, TCKN (opsiyonel), Doğum Tarihi, Departman, Pozisyon, İşe Giriş Tarihi, E-posta, Telefon. Kullanıcı hesabı ile ilişkilendirme (kayıt oluşturulurken hesap da açılabilir veya sonradan ilişkilendirilebilir).
- **Çalışan detay ekranı:** Temel bilgiler, izin geçmişi ve çalışana ait notlar (HR veya Yönetici tarafından girilen basit notlar).

5.3. İzin Yönetimi

5.3.1. Çalışan / Stajyer tarafı

- "İzin Talebi Oluştur" ekranına gidebilmeli.
- İzin türü (Yıllık, Ücretsiz vb.), başlangıç tarihi, bitiş tarihi ve açıklama alanlarını doldurabilmeli.
- Oluşturduğu talepleri listeden görebilmeli: durum (Beklemede / Onaylandı / Reddedildi), tarihler ve tip.

5.3.2. HR ve Yönetici tarafı

- "İzin Talepleri Listesi" ekranında bekleyen talepleri görebilmeli.
- Her talep için Onayla / Reddet yapabilmeli; reddetme durumunda opsiyonel açıklama yazabilmeli.
- Onaylanan/reddedilen izinler daha sonra raporlanabilmeli (en basit haliyle filtrelenebilir liste).

5.4. Stajyer Yönetimi (Özel Modül)

Proje özellikle bir stajyer tarafından geliştirilmek üzere kurgulandığından, stajyerleri takip edebileceğimiz basit bir modül eklenmesi istenir.

- **Stajyer listesi ekranı:** Ad, Soyad, Üniversite, Bölüm, Sınıf, Staj Başlangıç-Bitiş Tarihi, Mentor (yönetici) bilgileri.
- **Stajyer detay ekranı:** Temel bilgiler; staj süresince verilen görevlerin kısa listesi (elle girilen basit text alanı yeterli); mentor notları (ör. haftalık kısa geri bildirimler).

5.5. Dashboard ve Basit Raporlama

Giriş sonrası rol bazlı basit bir ana ekran (dashboard) beklenir:

- **HR Dashboard:** Toplam aktif çalışan sayısı, bekleyen izin talebi sayısı, aktif stajyer sayısı.
- **Yönetici Dashboard:** Kendi ekibindeki çalışan sayısı, ekibinden gelen bekleyen izin talepleri.
- **Çalışan/Stajyer Dashboard:** Kalan yıllık izin gün sayısı (manuel bir alan da olabilir), son 5 izin talebi.

6. Non-Fonksiyonel Gereksinimler

6.1. Kod Kalitesi ve Mimari

- Clean Architecture prensiplerine uyum: bağımlılık yönleri doğru kurgulanmalı.

- İş mantığı controller içinde değil, Application/Domain katmanlarında olmalı.
- Temiz ve anlamlı isimlendirmeler (sınıf, metot, değişken adları açık olmalı).
- Yorum satırları yalnızca gerçekten ihtiyaç duyulan yerlerde kullanılmalı.

6.2. Validasyon

- Form girişlerinde temel validasyon yapılmalı.
- Application katmanında use-case bazlı validasyonlar (ör. izin başlangıç tarihi bitiş tarihinden büyük olamaz).

6.3. Hata Yönetimi

- Kullanıcıya gösterilen hata mesajları sade ve anlaşılır olmalı.
- Global exception handling için basit bir middleware/filter kullanılabilir (opsiyonel ama teşvik edilen).

6.4. Güvenlik

- Şifreler hash'lenmiş tutulmalı.
- Login olmadan erişilememesi gereken tüm ekran/endpoint'ler için [Authorize] vb. kontroller eklenmeli.
- Rol bazlı [Authorize(Roles = "...")] kullanımı teşvik edilir.

7. Teslimatlar

- 1 Kaynak kod (Git repo linki).
- 2 README (kurulum & çalıştırma).
- 3 Mimari dokümanı (kısa): katmanlar ne işe yarıyor, hangi projede ne var, örnek akış ("Çalışan izin talebi nasıl işleniyor?").
- 4 Veri modeli / ER diyagramı.
- 5 En az 3-5 use case için unit test (zorlayıcı kısım).

Örnek test senaryoları:

- İzin başlangıç tarihi > bitiş tarihi olduğunda hata dönmesi.
- Stajyer eklerken zorunlu alanlar dolu değilse validasyon hatası.

Not: Bu doküman, gönderilen özgün gereksinim metninin temiz biçimlendirilmiş halidir. İçerik değiştirilmemiştir; yalnızca bölüm ve madde numaralandırması tutarlı olacak şekilde düzenlenmiştir.